

A Privilege Creep-Aware Role Mining Method for Enhanced Access Control Security

Vincent Bittard, Rim Ben Salem, Ahmed Bouzid,

Sara Imene Boucetta, Frédéric Cuppens, Nora Cuppens-Boulahia

Department of Software and Computer Engineering, Polytechnique Montréal, Montréal, Canada

{vincent.bittard, rim.ben-salem, ahmed.bouzid, sara-imene.boucetta, frederic.cuppens, nora.boulahia-cuppens}@polymtl.ca

Abstract—Role Mining (RM) extracts Role-Based Access Control (RBAC) structures from user-permission assignments to reduce administrative overhead. However, existing approaches usually make the assumption of clean datasets, while real-world systems suffer from anomalies like privilege creep, the gradual accumulation of unnecessary permissions.

The proposed approach aims to detect potential privilege creep users who should be reviewed first, and identify legitimate permissions assignments to be expressed in RBAC, reducing management complexity. It consists of a two-step procedure: clean the User-Permission Assignment matrix (UPA) using a clustering and statistical analysis, then build an RBAC state using a regular role mining algorithm.

The proposed approach yields an average of 90% in privilege creep detection accuracy and over 95% privilege creep correction, evaluated on synthetically made datasets. Evaluation on real-world datasets demonstrates an average 4-fold reduction in required roles while maintaining at least 80% UPA coverage.

Index Terms—Role mining, RBAC, access control, clustering, anomaly detection

I. INTRODUCTION

To alleviate Identity and Access Management (IAM) costs, system administrators often structure permission assignments using RBAC, grouping users with similar need-to-know. This enables a more scalable and maintainable access control configuration. Recent approaches to role mining have focused on methods designed for clean datasets. In real-world access control systems, this assumption rarely holds. Over time, the quality of IAM systems degrades, with privilege creep representing a key anomaly in insider threat contexts: an incremental accumulation of permissions resulting from job transitions, organizational changes, temporary assignments etc. While this issue can be manually addressed in small organizations, the complexity increases significantly in larger enterprises, especially since permission review is often fragmented and delegated.

The process addressing this issue is known as Noise Role Mining [1], [2], [3], [4]. Noise definitions commonly fall under two categories : administration errors (wrongly granted or revoked permissions, regardless of cause) or applicability exceptions (legitimate but policy-complicating permissions). Therefore, noise-aware role mining techniques can be viewed as a subset of privilege creep-aware techniques, aiming to remove noisy assignments directly, with the goal to improve security or reduce management costs.

Historically, first approaches focused on mining roles directly on noisy data. Vaidya et al. [1] introduce the δ -Role Mining Problem (RMP) and Minimal Noise RMP to fulfil this goal, which allow partial UPA expression by tolerating mismatches between the source UPA and the mined one, effectively removing noise. Multi-Assignment Clustering (MAC) by Frank et al. [5] similarly allows partial expression, and clusters users with overlapping permissions, discarding low-probability assignments to remove noise.

Other approaches use a 2-step method: clean the data first and mine later. Molloy et al. [2] use binary matrix decomposition algorithms for the cleaning step. Singular Value Decomposition (SVD), Non-Negative Matrix Factorization (NMF), Binary Non-Negative Matrix Factorization (BNMF) and logistic Principal Component Analysis (PCA) are compared to δ -RMP and MAC. The UPA is decomposed and reconstructed in binary form via inverse transformation, using a step function to enforce binary values. Since the decompositions are not exact, some data is destroyed, hence removing noise. The evaluation by Molloy et al. [2] demonstrates that the 2 step-method yields better noise-removing results on synthetic datasets than the other previously mentioned approaches.

The main contributions of this paper are:

- A new parameterizable synthetic data generator that aims to build realistic RM datasets, able to inject noise and privilege creep related to common enterprise scenarios.
- A proposed approach to clean RM datasets by removing potential noise and detecting instances of privilege creep, in order to mine roles on the cleaned dataset.
- A thorough performance evaluation introducing new metrics to assess the accuracy of the privilege creep removal.

The remainder of this paper is structured as follows: Section II reviews recent approaches relevant to the problem addressed in this paper. Afterward, Section III and IV define the basis of RM, the terminology, and the nomenclature for noise and privilege creep used in this paper. Then, the proposed privilege creep-aware role mining approach is explained in section V, while the synthetic dataset generation method is described in section VI. Next, section VII sets the evaluation metrics and benchmark for validation. Section VIII then presents results on synthetic and real-world datasets. Finally section IX provides a discussion about the advantages, shortcomings, and potential improvements of the proposed method.

II. RELATED WORK

A. Recent role mining approaches

Kang et al. [4] improve previous noise RM algorithms using role interpretability. The mined role set is given a user attribute-based interpretability score to minimize alongside reconstruction error. This produces roles that more appropriately fit business needs, ultimately used to reduce management overhead. While the issue of noise is tackled explicitly in this work, its nature remains unclear and privilege creep is not directly addressed as the algorithm goal is to approximate existing access patterns.

Durdag and Coskuncay [6] choose to reconfigure RBAC systems by clustering similar roles based on permission similarity using Agglomerative Hierarchical Clustering (AHC). Identified clusters serve as reference structures to support system redesign or cleanup, improving manageability of complex RBAC configurations. Their approach removes problematic data during preprocessing and validates against expert expectations, but lacks mechanisms to identify or correct anomalous permission patterns.

Wang et Wu [7] propose a method to reduce role proliferation in RBAC systems using formal concept analysis and concept lattice factorization. The method first generates an initial state, then optimizes by balancing user-role and permission-role assignments while reducing the concept lattice dimensionality. This allows the establishment of role mining objectives before algorithm execution. However, the authors note needs for benchmarking, scalability improvements, and noise robustness testing against existing methods.

Nobi et al. [8] introduce Deep Learning Based Access Control (DLBAC), neural networks that learn directly from raw user and resource metadata, eliminating the need for manual engineering of roles, attributes, and policies. The prototype DLBAC- α demonstrates superior accuracy and generalization compared to classical policy mining and machine learning approaches, while addressing explainability concerns through interpretation techniques like Integrated Gradients. However, DLBAC does not directly tackle privilege creep because it learns from existing authorization data that may already contain anomalous access rights. This limitation is acknowledged by the authors, stating that errors in the datasets used could introduce bias in the trained model.

The work in this paper differs from the recent approaches by explicitly modeling privilege creep scenarios separately from generic noise, and evaluating performance based on the algorithm's ability to structure legitimate permissions while identifying and removing excessive permissions.

B. Recent privilege creep detection approaches

Parkinson et al. [9] present an unsupervised tool to detect privilege creep instances in file system Access Control Lists (ACL) using χ^2 statistics, establishing an average 96% accuracy in privilege creep detection on synthetic datasets. The approach is scientifically sound but suffers from a key evaluation flaw: using accuracy instead of F-measure artificially

inflates performance since legitimate users greatly outnumber anomalous instances, causing high true negative counts to skew the metric despite an average 30% false negative rate in detecting actual privilege creep. Parkinson et al. [10] also propose a fuzzy logic-based approach to identify critical privilege creep in access control policies by modeling user trust, resource sensitivity, and permission power as fuzzy sets rather than binary classifications. This new approach is reliant on security event logs, producing better results than the Creeper tool but still suffering from the same evaluation framework flaw.

Alexander and Chikwari [11] propose a graph-based AI framework that models enterprise IAM as a knowledge graph, applying Graph Neural Networks (GNNs), community detection algorithms (Louvain method), and graph clustering to discover latent role structures from access patterns, while using Graph Autoencoders and Isolation Forests to detect privilege creep and anomalous permissions. Their approach claims to reduce role redundancy by 38% and achieve 93.5% precision in anomaly detection. However, critical limitations include evaluation exclusively on synthetic datasets, and comparison against an inappropriate baseline using k-means, unsuited for anomaly detection, rendering the reported F1-score improvement from 74.8% to 91.3% potentially misleading.

This paper diverges by introducing an unsupervised role mining approach independent of user attributes or event data, dedicated evaluation metrics, and a parameterizable synthetic generator that realistically injects privilege creep, addressing evaluation limitations and generic noise models in prior work.

III. DEFINITIONS

The National Institute of Standards and Technology (NIST) [12] formally defines the common role mining context:

- Let U, P the set of users and the set of permissions respectively
- Let $UPA \subseteq U \times P$ the binary user-permission assignment matrix, a many-to-many mapping.

The NIST also formalized the Basic Role Mining Problem (RMP) [12] as a binary matrix decomposition problem:

Given the common role mining context, find a set of roles R and two binary matrices $UA \subseteq U \times R$, the user-to-role assignment matrix and $PA \subseteq P \times R$, the permission-to-role assignment matrix where $UPA = UA \times PA$ minimizing $|R|$.

In this paper, a permission refers to an existing column in the UPA matrix, a user refers to an existing row in the UPA matrix, and a permission assignment is defined as an existing mapping in the UPA matrix. The concept of legitimate permission is defined as permission assignments that should be mined during the RM process, as opposed to noise or privilege creep that should not be mined.

IV. IDENTIFYING NOISE AND PRIVILEGE CREEP

A. Noise

The proposed nomenclature draws inspiration on the types of noise identified by Molloy et al. [2] and Vaidya et al. [1]:

- Correctness noise: isolated administration errors that usually occur when a user goes through access provisioning for the first time, only affecting a small set of permissions. Correctness noise is usually additive, meaning additional permission assignments on affected users, because of the availability issue subtractive noise causes.
- RBAC applicability noise: legitimate permissions that are not sufficiently shared amongst users to be usefully expressed into RBAC. Indeed, expressing them would increase the complexity of the RBAC state and therefore undermine the management advantages of maintaining this structure for access control.

B. Privilege creep

Two privilege creep scenarios are considered:

- Scenario 1: an employee moves to a different position within the organization but retains a subset of permission assignments from their previous duties due to an incomplete deprovisioning. Let's call this type I privilege creep.
- Scenario 2: a group of employees was assigned to a now finished temporary project. The additional permissions they were given to fulfil their responsibilities have not been revoked entirely. This can occur due to unmaintained records of projects. Let's call this type II privilege creep.

The key difference between these two privilege creep types is the subset of permissions impacted.

1) *Type I*: The impacted permissions are still legitimate for other employees. Indeed, those currently in the same position as the privilege crept employee previously held, or the employee who takes the new vacant position, have a valid need for these permissions to fulfil their task.

2) *Type II*: The impacted permissions are not legitimate for employees outside the project. Indeed, the permissions are project-bound and once the project ends, no employee should retain them.

V. PRIVILEGE CREEP-AWARE ROLE MINING

As explained in section I, the proposed method uses a 2-step approach : clean the dataset and then mine the roles with a regular role mining algorithm.

A. Prerequisite

Translate the user-permission assignments into a standard binary UPA matrix X as defined in section III.

B. Dimensionality reduction

The first step is to use a dimensionality reduction algorithm. This is done using Truncated-SVD (TSVD). TSVD offers faster computing times on sparse matrices, does not require centered data, and its partial decomposition of X reduces the impact of noise [2]. Indeed, the assumption that UPA matrices are sparse usually holds true [1] [2] [13] [3].

Let $k \in \mathbb{N}^*$ the rank of the TSVD decomposition of X , where k represents the dimensionality of the reduced space.

The low-dimensional embedding Z of X is obtained by the following equation :

$$Z = XV_k^T \quad (1)$$

Where V_k is the top k right singular vectors matrix of X .

For each component $i \in \llbracket 1, k \rrbracket$, Z_i the i -th column of Z . The empirical variance of the i -th component corresponds to the variance of Z_i :

$$Var(Z_i) = \frac{1}{n} \sum_{j=1}^n (Z_{ji} - \mu_i)^2, \text{ where } \mu_i = \frac{1}{n} \sum_{j=1}^n Z_{ji} \quad (2)$$

The variance of X is computed with:

$$Var(X) = \sum_{j=1}^d Var(X_j) = \sum_{j=1}^d p_j(1 - p_j) \quad (3)$$

Where $p_j = \frac{1}{n} \sum_{i=1}^n X_{ij}$ is the fraction of ones in column j . The total explained variance ratio R_k is computed using the explained variance ratio r_i for each component $i \in \llbracket 1, k \rrbracket$ using the following equation:

$$R_k = \sum_{i=1}^k r_i = \sum_{i=1}^k \frac{Var(Z_i)}{Var(X)} \quad (4)$$

The goal is to find a TSVD rank k that renders around 80% total explained variance ratio [2]. Since this sole condition can produce an overwhelmingly large number of components k for the next step in the approach, another condition is added on r_k . The search stops when the following condition is met :

$$R_k > 0.80 \text{ OR } r_k < \epsilon \quad (5)$$

At this point, adding more and more components yields diminishing returns. Empirically, setting $\epsilon = 0.02$ provides reliable results.

C. Clustering and outlier identification

Z should contain less noise than X and consists of real valued components that capture the most important user-permission relationships. Hence, clustering is performed on the low-dimensional embedding Z .

Since the primary objective of the proposed method is to detect privilege creep, choosing a clustering algorithm capable of identifying anomalous users as outliers becomes necessary. The selected clustering algorithm to fulfil this goal is Density-Based Spatial Clustering of Applications with Noise (DBSCAN), which offers easily interpretable parameters to be determined :

- min_{points} the minimum number of points required to form a cluster. Since clusters should identify users who present similar permissions, which usually occurs in teams of employees working together, min_{points} is set to the smallest team size detectable in the UPA.
- ϵ the radius of the neighborhood with respect to some point. To determine an ideal value for ϵ the 1 Nearest-Neighbor (1-NN) Euclidean distance of all users are

computed and plotted in ascending order. Then, ϵ is set to the cutoff value corresponding to the elbow of the curve. In the proposed approach, the elbow is detected automatically using the normalized difference curve technique. The ϵ value can still be overwritten manually using the visual method.

Let's call C , the user-cluster assignment table giving the DBSCAN labels for each user including the outliers.

D. Cleaning and role mining

In order to clean the UPA C and X are used together by performing a statistical analysis on each cluster to determine which permissions assignments are to be expressed. Outliers are first removed off of X to ensure role quality. Permission prevalences are then computed for each cluster, defined as the share of users within a cluster who possess a given permission. Using a threshold t_c the permissions with prevalence less than t_c are removed and the permissions with prevalence greater than t_c are kept. The matrix obtained after the cleaning process is called the cleaned UPA K .

E. Role mining

FastMiner [14] and Optimal Boolean Matrix Decomposition using BasicRMP introduced by Lu et al. [15] are used as the role mining algorithm with a greedy approach. The algorithm is run on K with the outliers removed. It expresses the assignments entirely without approximations. After the role mining is done, outliers need to be reintroduced into K .

F. Reunification

This step is a new addition from previous methods in the literature that do not identify outliers. Two use cases are considered:

- 1) Omniscient reunification: This is the case used for synthetic datasets, adding the outliers back into the cleaned UPA K with their known legitimate permissions. This emulates the presence of an expert who is able to clean the privilege creep and noise out of a user permission pool perfectly.
- 2) Heuristic reunification: This is the case in which the proposed tool will most likely be used in an organization setting to provide recommendations for outliers. Two concurrent approaches are used. The first one is to assign mined roles to the outliers even though all their permissions may not be covered. This gives a rough idea of potential roles this user could have, and also brings out the potentially problematic permissions that are not covered. The second approach uses the information on clusters: The barycenter of every cluster using the binary coordinates of users is computed. Then, every outlier is linked to the closest barycenter. This gives additional insight on which cluster the identified outlier could be part of and their potential legitimate permissions. Ultimately the decision should be made by an administrator in charge of reviewing user accounts as both approaches could provide conflicting information.

The proposed approach is evaluated using the omniscient reunification. Thus, the metrics chosen in section VII-C account for the accuracy of outlier detection. The following section details the generation of synthetic datasets, which is another contribution that this paper brings forth.

VI. SYNTHETIC DATASETS

Given that the proposed approach is evaluated on privilege creep detection and noise correction, and that existing synthetic dataset generators in the literature do not account for privilege creep, a new dataset generation method is required. One of the main contributions of this paper is a flexible generator with adjustable parameters to produce synthetic datasets mimicking real-world ones.

A. Generating legitimate permissions

To build the generator, inspiration was taken from the Tree-Based Data Generator from Molloy et al. [16]. The main changes are made on the propagation strategy to generate trees, the process of adding noise, and the new process of adding privilege creep instances. The proposed generator produces a mock business organization hierarchy structure using a tree.

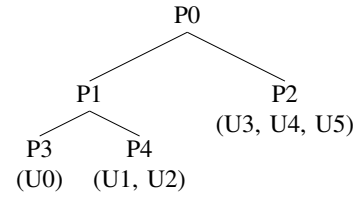


Figure 1: Template example; U1,U2 inherit permission sets P0, P1 and P4, U0 inherits permission sets P0, P1, P3 etc.

The tree structure, assigned with permissions and users, is referred to as the template. As a RM objective, mined roles should correspond to the template structure. Knowing this, templates are used to create legitimate permission hierarchies.

The generator starts with the root node and then grows child nodes iteratively, using six parameters to build trees:

- `min_depth` and `max_depth`, encode the minimum and maximum depth of the tree, nodes stop propagating between these bounds with a linearly decaying probability.
- `min_children` and `max_children` encode the minimum and maximum number of child nodes allowed per node.
- `avg_branch` and `std_dev` encode how nodes propagate in the tree. The number of child nodes follows a normal distribution centered around `avg_branch` with a standard deviation of `std_dev`, brought back to integers.

Given a target number of permissions and users, a random number of permissions is assigned to every node using a binomial law. A random number of users is assigned to leaves of the tree using the same binomial law method. Users then inherit the permissions of the leaf node they are assigned to, and all permissions assigned to parents nodes above them (see example on figure 1). This process generates the legitimate UPA matrix.

B. Adding privilege creep

As identified in section IV, two types of privilege creep are added to the UPA matrix: type I and type II privilege creep. Three parameters are introduced for this purpose:

- 1) p_{PC} the portion of users who present cases of type I privilege creep.
- 2) c the portion of permission assignments copied from another user in a type I privilege creep instance.
- 3) r the number of added permissions for users who present type II privilege creep. All affected users are then granted these permissions.

In order to always have a varied set of privilege creep instances the following rules are used:

- 30% of type I privilege creep instances use $c = 1$
- 70% of type I privilege creep instances are assigned linearly decreasing values of c from 1 to 0.
- The number of users in each type II privilege creep instance is fixed to 8, and the number of instances is computed using this formula:

$$n_{\text{Type II instances}} = \lfloor \log_{10}(n_{\text{users}}) \rfloor \quad (6)$$

Thanks to these parameters, it is possible to produce privilege creep instances of varying frequency and magnitude. The privilege crept UPA is then created by adding the privilege crept permissions assignments to the legitimate UPA.

C. Adding noise

To make the dataset more realistic, noise is added to the privilege crept UPA. This follows the discussion in section IV on noise identification. RBAC applicability noise is added using the first two parameters and correctness noise using the third parameter :

- p_{noise} : noise percentage denotes the ratio of noisy permission assignments to be added to the UPA matrix, expressed as a proportion of the number of legitimate assignments.
- d_{noise} : noise density denotes how dense the added assignments are. The number of distinct added permissions N_{noisy} must be computed to match the noise density parameter. This is done using the formula:

$$N_{\text{noisy}} = \frac{N_{\text{legit}} p_{\text{noise}}}{d_{\text{noise}} n_{\text{users}}} \quad (7)$$

Where N_{legit} is the number of legitimate assignments. The added noise assignments are then generated with a Bernoulli experience on a matrix of size $(n_{\text{users}}, N_{\text{noisy}})$ with $p = d_{\text{noise}}$, concatenated to the privilege crept UPA.

- $p_{\text{legit-noise}}$, noise percentage on legitimate permissions indicates the ratio of additional assignments to introduce relative to the number of already injected noisy permissions. These assignments are uniformly distributed on the legitimate permission matrix directly, only flipping zeros into ones.

VII. EVALUATION

The synthetic dataset evaluation framework is first defined. Then, the real-world datasets used for evaluation are mentioned. Finally, Metrics are listed at the end of the section.

A. Synthetic dataset benchmark

Since it is impossible to test every parameter combination with the proposed dataset generation method, standard configurations are defined for a variety of problem sizes, general hierarchy aspect of the legitimate templates, and privilege creep & noise distributions using profiles:

- Table I defines profiles used to build template trees (section VI-A), selected to have a variety of depth and branching, mimicking different organizational structures.
- Table II defines profiles used to add noise and privilege creep to the legitimate UPA (section VI-C), selected based on the assumption that the number of privilege creep instances increases with higher noise levels.
- Table III defines profiles used to assign users and permissions to tree nodes (section VI-A), selected to restrict the number of permissions assigned to nodes and the number of users assigned to leaves.

Evaluation on noise levels (2), spans across the parameter profiles from tables I and II, using the default tension parameters from table III. Evaluation under tension (3), spans across the parameter profiles from tables I and III, using the default noise parameters from table II.

Name	children		depth		avg_branch	std_dev
	min	max	min	max		
large_flat	2	4	2	4	3	1.5
small_flat	1	5	1	3	3	1
large_string	1	2	10	15	1.7	0.5
small_string	1	2	5	8	1.6	0.4
binary_tree	1	3	2	5	2	0
highly_random	1	6	2	5	2	2

Table I: Profiles for generating legitimate permission trees

Name	Acronym	Noise			
		percent	density	legit	PC
no noise no pc	NN	0	N/A	0%	0%
low noise, low density	LNLD	5%	1%	10%	3%
high noise, low density	HNLD	15%	1%	15%	5%
low noise, high density	LNHD	5%	4%	15%	5%
high noise, high density	HNHD	15%	5%	20%	8%
default	/	15%	2%	10%	3%

Table II: Profiles for noise and privilege creep levels

Name	Acronym	User		Permission	
		min	max	min	max
no tension	NT	15	25	10	40
tension on permissions	TP	15	25	2	6
tension on users	TU	2	8	10	40
tension on both	TUTP	2	8	2	6
default	/	15	25	15	45

Table III: Profiles for tension on users and permissions

B. Real-world dataset benchmark

The real-world datasets provided by Ene et al. [17] are used for the evaluation. Despite their publication in 2008, these datasets have become established benchmarks in the

literature and continue to be referenced in recent studies for performance comparison on real-world instances [4], [18], [19], [20], [21]. These are: *americas_large*, *americas_small*, *apj*, *customer*, *domino*, *emea*, *firewall1*, *firewall2*, *healthcare*.

C. Metrics

The main goal of the chosen metrics is to measure how accurate the privilege creep detection and correction is without compromising permissions flagged as legitimate.

- 1) Legitimate Permission Retention (LPR):
The F1-score computed between the cleaned UPA and the reference legitimate UPA, excluding potential outliers. It reflects the accuracy of legitimate permission recovery, while minimizing the retention of noise.
- 2) Permission Expression (PE):
The number of permission assignments retained in the cleaned UPA divided by the number of assignments in the noised UPA. An effective cleaning process would result in a permission expression value that is close to the one computed on the legitimate matrix.
- 3) Privilege Creep Correction (PCC):
The percentage of privilege crept assignments still present in the cleaned UPA. A value close to 1 reflects the effectiveness of the cleaning process.
- 4) Privilege Creep Detection Accuracy (PCDA):
The F1-score computed between the set of identified outliers and the ground truth set of anomalous users. It measures the accuracy of the anomaly detection.
- 5) Role Count Deviation (RCD):
The difference between the number of ideal roles, identified to be the number of leaves in the template (see section VI-A) and the number of mined roles divided by the number of ideal roles.
- 6) Runtime metric:
The runtime of the cleaning process in seconds.

On synthetic datasets, the proportion of identified privilege creep instances is also recorded by type.

VIII. RESULTS

min_{points} is set to 5 to ensure clusters represent meaningful user groups (teams/departments) as explained in section V-C. Empirical testing has shown that lower values produce fragmented clusters where the statistical threshold t_c incorrectly removes legitimate permissions, as small clusters exhibit high variance in permission prevalence.

A. Performance using synthetic datasets

The results are given as box plots. Each plot renders the distribution of a given metric evaluated on 20 runs of each structure parameter defined on table I. Given that there are 6 structure parameters, each box plot contains 120 different samples. Metrics are explained in section VII-C.

1) *Experiment with varying noise levels*: Figures 2a and 2b concurrently show that legitimate permission assignments were accurately retrieved, avoiding the expression of noisy assignments. Indeed, the median LPR is above 97.5% on all noise levels and permission expression shows a proportional decrease with increasing noise levels, as expected. About privilege creep, conflicting information seem to stem from figures 2c and 2d: the detection accuracy hovers around 80%, sometimes reaching values below 50% on the worst runs, while almost all privilege crept assignments are corrected with median PCC values at 100% across all noise levels. This occurs as type I privilege creep instances are accurately detected most of the time, whereas type II instances are more difficult to detect. The statistical analysis cleaning process then removes undetected type II privilege creep permission assignments. Figure 2e reveals that the proposed approach tends to produce fewer roles than expected VI-A, probably due to legitimate permissions being erased by the cleaning process. The average runtime for the cleaning process is below 5 seconds, never exceeding 1 minute as figure 2f demonstrates. Longer runtimes can be attributed to exceptionally large datasets. When the datasets are free from noise and privilege creep, figures 2b, 2a and 2e confirm that the approach accurately retrieves legitimate assignments with minimal false positive outlier occurrences, and produces the expected amount of roles on average.

2) *Experiment with varying tension levels*: Figures 3a and 3b demonstrate that legitimate permissions assignments are generally not accurately retrieved when tension is put on users. Indeed, LPR plummets to 0% on several of these datasets and PE is lower than expected, reaching values that do not correspond to the amount of noise, meaning legitimate assignments have been removed massively. Regarding privilege creep, detection accuracy decreases by 30% when tension is applied to users, with many false positives as shown in figure 3d. All of this can be explained when too few users are present in the UPA matrix to form large significant groups: the cleaning process removes larger amounts of permissions due to wrongly identified clusters, which also causes a lot of users to be wrongly flagged as outliers. These effects propagate to the role mining step, no roles can be mined when great amounts of permissions have been removed. This is shown in figure 3e where runs under user tension produce half the roles that would normally be needed to express the assignments on average. Cleaning runtimes are faster than the previous experiments, averaging 3 seconds due to the datasets reduced size under user and/or permission tension.

B. Performance using real-world datasets

On real-world datasets, hyperparameters are determined through a semi-automated process combining the algorithmic methods from sections V-B and V-C with manual refinement. The automated procedures provide initial recommendations for n_{comps} , ϵ , and t_c . These initial values are then manually adjusted based on the following criteria: (1) maximizing the explained variance ratio R_{var} while respecting the diminishing

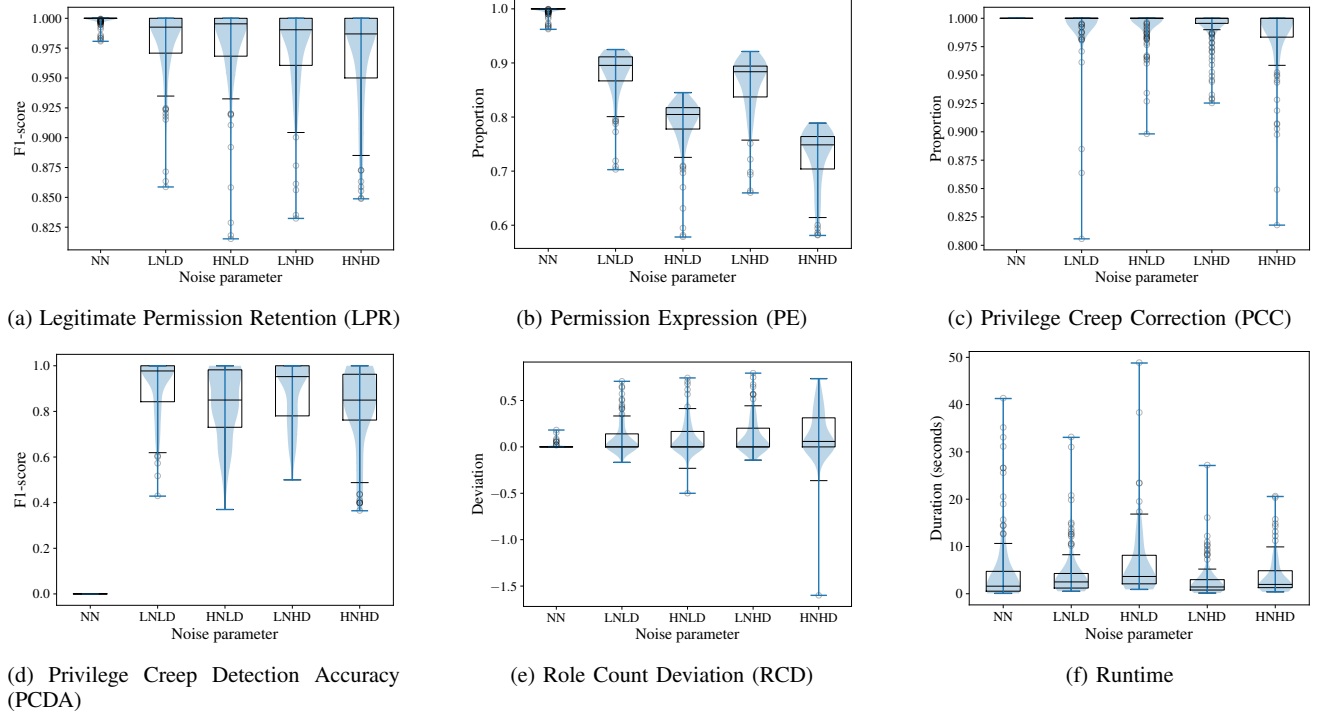


Figure 2: Performance analysis against noise levels

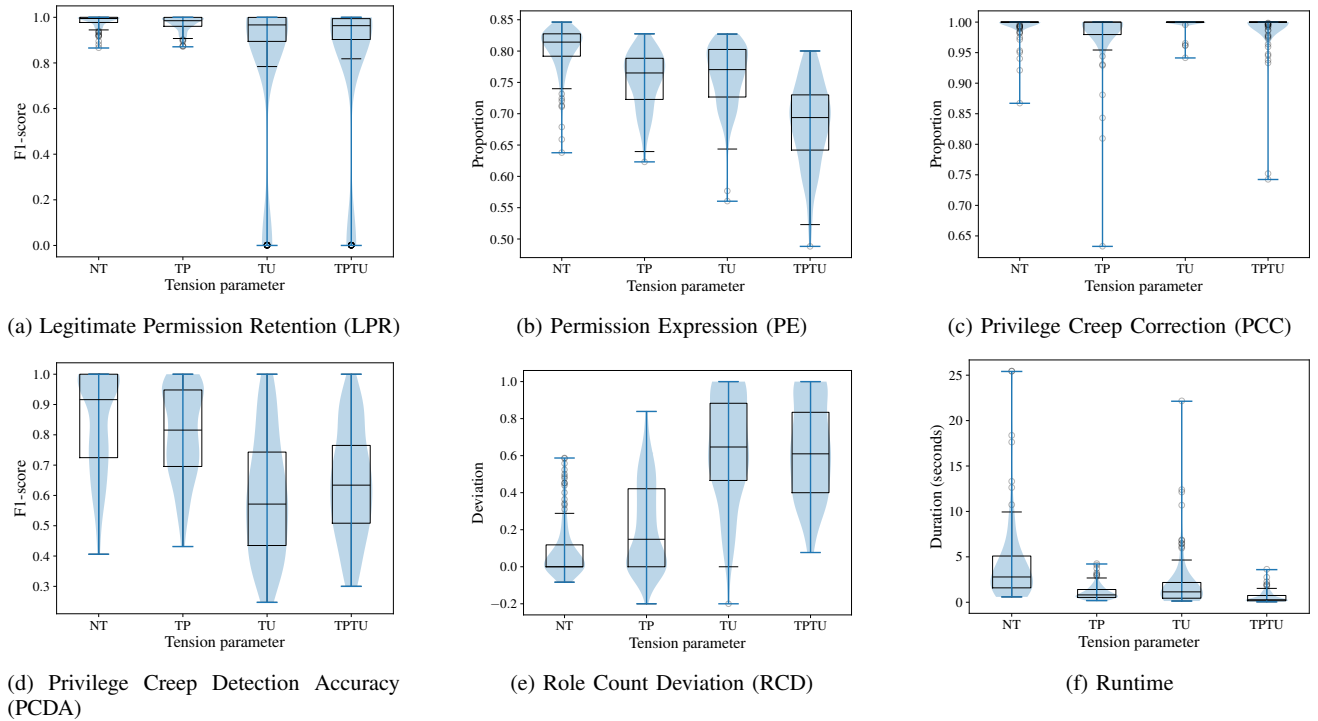


Figure 3: Performance analysis against tension levels

returns threshold described in section V-B, (2) ensuring stable cluster formation that effectively separates users with irregular permissions patterns from the others using T-distributed stochastic neighbor embedding (t-SNE) projections, and (3) optimizing the trade-off between permission expression coverage and noise removal through t_c calibration, guided by the established 80-20 principle in access control management. For each dataset, multiple parameter configurations near the automated recommendations were evaluated, and the combination yielding the best balance between UPA coverage and role quality was selected. Final hyperparameter values and corresponding R_{var} are reported in Table IV.

Dataset	$ U $	n_{comps}	R_{var}	ϵ	t_c
americas_large	3485	10	0.646	2	0.3
americas_small	3477	7	0.807	1	0.5
apj	2044	6	0.350	0.5	0.005
customer	10021	12	0.500	1	0.02
domino	79	4	0.840	1	0.1
emea	35	12	0.815	16	0.05
fire1	365	3	0.882	0.3	0.5
fire2	325	1	0.832	0.03	0.5
healthcare	46	3	0.841	1.5	0.4

Table IV: Determined hyperparameters

Table V compiles the results of the approach. The optimal number of roles used for comparison is provided by HP Labs [17] and reported by Blundo et al. [19]. Π represents regular permission expression, and π permission expression computed without outliers. The number of outliers is also reported.

Dataset	Π	π	no. out.	n_{roles}	$n_{optimal}$
americas_large	0.755	0.841	45	98	398
americas_small	0.881	0.913	31	21	178
apj	0.727	0.759	13	183	453
customer	0.798	0.812	58	47	276
domino	0.171	0.919	7	6	20
emea	0.615	0.799	4	25	34
fire1	0.905	0.974	14	4	64
fire2	0.994	1	7	1	10
healthcare	0.881	0.946	4	2	14

Table V: Results on real-world datasets

First, *americas_large*, *apj* and *customer* did not reach above 80% permission expression probably because of their sheer size and noise levels. Therefore, they required lower than average t_c to be expressed appropriately. The proposed approach yields over 75% permission expression on all datasets. Looking at the difference between Π and π , it is clear that on datasets like *domino* or *emea*, the small pool of identified outliers concentrate the majority of permission assignments, since π is above 80%, meaning clustered users have 80% of their permissions expressed. These users are likely administrators with an exceptionally high number of permission assignments, and therefore they were flagged as outliers. Overall, the number of identified outliers is between 1% and 10% of the number of users for all datasets, with a manageable absolute number that never exceeds 60 users. Also, the approach produced fewer roles across all datasets compared to the optimal number used for full expression, effectively dividing the number of roles by

4 on average. The exceptionally low amount of roles produced on *fire1* and *fire2* is likely due to a great number of similar users with a handful of exceptions.

IX. DISCUSSION AND CONCLUSION

This paper addresses the issue of role mining in the presence of privilege creep through three main contributions:

- A parameterizable synthetic data generator that aims to create realistic role mining datasets, with controlled noise and privilege creep injection.
- An effective cleaning methodology that removes noise and detects privilege creep in UPA matrices, achieving 90% detection accuracy and 95% legitimate permission retention on average.
- A comprehensive evaluation framework providing new metrics (LPR, PCC, PCDA, RCD) specifically designed for privilege creep-aware role mining.

A. Performance and practical impact

The proposed approach demonstrates strong performance on several datasets. On *americas_small*, *fire1*, *fire2*, and large synthetic datasets without user tension, the method expresses 90% of permission assignments while accurately removing noise and privilege creep. Notably, the approach reduces the number of required roles by up to a factor of ten, significantly improving access control manageability in large organizations. A key advantage of the proposed approach is its security-preserving nature [3], as no new permission assignments are added to the UPA, making it suitable for critical access review. The reunification phase improves role quality by removing potentially anomalous users out of the role mining phase.

Regarding privilege creep detection, Type I instances are detected with an accuracy above 90%, demonstrating the method's effectiveness for this common scenario. Type II privilege creep proves more challenging to detect as noise levels increase, with PCDA scores frequently falling below 50%. This occurs because higher noise levels render Type II privilege creep permission assignments statistically indistinguishable from random noise. However, due to the statistical analysis process used to clean the dataset, Type II privilege creep permissions are typically removed during the cleaning phase, even when not explicitly detected as privilege creep. While this maintains the cleaning objective, it highlights a limitation in the detection of project-based anomalies.

B. Limitations and areas for improvement

Hyperparameter sensitivity: The method's performance depends on manually tuning key parameters (k , ϵ , t_c). While heuristic estimates are provided for k and ϵ , no such guidance exists for t_c . This limitation is particularly evident on real-world datasets such as *apj*, *customer*, *domino*, and *emea*, which required unexpectedly low cleaning thresholds ($t_c < 0.3$) to achieve permission expression scores above 50%. This behavior stems from the interdependence of ϵ and t_c : targeting an outlier rate below 10% often necessitated inflated ϵ values, which caused cluster to merge and consequently

required lower-than-typical t_c thresholds to maintain adequate cleaning. These observations reveal a cascading effect amongst hyperparameters, with ϵ and especially t_c exhibiting the highest sensitivity. Visualizations of clusters with tools like t-distributed stochastic neighbor embedding (t-SNE) could guide hyperparameters selection.

Performance degradation on small datasets: Both performance evaluations reveal an inherent limitation: the approach demonstrates reduced effectiveness on datasets with fewer than 100 users approximately, such as *domino*, *emea*, *healthcare*, and synthetic datasets with user tension. On synthetic datasets, privilege creep detection accuracy averages only 60%, and legitimate permissions are erroneously removed. This degradation stems from the statistical properties of the clustering approach: DBSCAN requires sufficient sample density to reliably distinguish meaningful clusters from outliers, and small user populations provide insufficient data points, therefore flagging more users as outliers. The statistical threshold t_c , becomes unreliable when clusters contain too few members, as small anomalies within clusters are disproportionately represented.

Synthetic dataset generation: The data generator is based on a hierarchical tree structure with binomial distribution for users and permissions on nodes, which produces cleaner datasets than real-world ones, and exhibit artificial structural patterns that may not reflect the complexity and irregularities of actual enterprise environments. As such, the evaluation results on synthetic data are to be interpreted with caution.

Limited baseline comparison: Evaluation on real-world datasets uses HP Labs ideal results to evaluate role number reduction. However, it lacks comparison with state-of-the-art role mining techniques. Future work should primarily benchmark permission retention and privilege creep detection against methods in the related work section.

C. Future work

On the approach itself, user attributes could be used to enhance the clustering process by providing additional context. To address the hyperparameter sensitivity issue, adaptive thresholds based on cluster characteristics should be developed. Additionally, exploring alternative clustering algorithms, such as Isolation Forest or Local Outlier Factor (LOF), may improve performance on small datasets.

The synthetic data generator needs to be extended to model matrix-style and network-style organizational architectures, as well as separation of duties (SoD) constraints, enabling more realistic test scenarios. A comparative assessment against state-of-the-art role mining approaches would complete the validation of the method relative to existing techniques. Finally, mechanisms to assess and mitigate the operational impact of permission removal should be developed, to ensure the method's practical deployability in production environments.

ACKNOWLEDGMENT

We extend our gratitude to MITACS, Banque Nationale, Desjardins, Mondata, and Qohash for their invaluable support and contributions to this research.

REFERENCES

- [1] J. Vaidya, V. Atluri, Q. Guo, and H. Lu, "Role mining in the presence of noise," in *IFIP Annual Conference on Data and Applications Security and Privacy*, Springer, 2010, pp. 97–112.
- [2] I. Molloy, N. Li, Y. Qi, J. Lobo, and L. Dickens, "Mining roles with noisy data," in *Proceedings of the 15th ACM Symposium on Access Control Models and Technologies*, 2010, pp. 45–54.
- [3] J. Crampton, E. Eiben, G. Gutin, D. Karapetyan, and D. Majumdar, "Generalized noise role mining," in *Proceedings of the 27th ACM Symposium on Access Control Models and Technologies*, 2022, pp. 91–102.
- [4] H. Kang, G. Liu, Q. Wang, Q. Zhang, J. Niu, and N. Luo, "An improved minimal noise role mining algorithm based on role interpretability," *Computers & Security*, vol. 127, p. 103 100, 2023.
- [5] M. Frank, A. P. Streich, D. Basin, and J. M. Buhmann, "Multi-assignment clustering for boolean data," *The Journal of Machine Learning Research*, vol. 13, no. 1, pp. 459–489, 2012.
- [6] O. Durdag and A. Coskuncay, "Reconfiguring role-based access control via role clustering," *IEEE Access*, 2025.
- [7] T. Wang and Q. Wu, "Role Minimization Optimization Algorithm Based on Concept Lattice Factor," *Mathematics*, vol. 11, no. 14, p. 3047, 2023.
- [8] M. N. Nobi, R. Krishnan, Y. Huang, M. Shakarami, and R. Sandhu, "Toward Deep Learning Based Access Control," in *Proceedings of the Twelfth ACM Conference on Data and Application Security and Privacy*, ser. CODASPY '22, Association for Computing Machinery, 2022, pp. 143–154.
- [9] S. Parkinson, S. Khan, J. Bray, and D. Shreef, "Creeper: A tool for detecting permission creep in file system access controls," *Cybersecurity*, vol. 2, no. 1, p. 14, 2019.
- [10] S. Parkinson and S. Khana, "Identifying high-risk over-entitlement in access control policies using fuzzy logic," *Cybersecurity*, vol. 5, no. 1, p. 6, 2022.
- [11] D. Alexander and D. K. Chikwari, "Graph-Based AI Techniques for Role Mining and Access Optimization in Complex Enterprises," *International Journal of Advanced Engineering Technologies and Innovations*, vol. 1, no. 3, p. 519, 2023.
- [12] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed nist standard for role-based access control," *ACM Transactions on Information and System Security (TISSEC)*, vol. 4, no. 3, pp. 224–274, 2001.
- [13] D. Zhang, K. Ramamohanarao, and T. Ebringer, "Role engineering using graph optimisation," in *Proceedings of the 12th ACM Symposium on Access Control Models and Technologies*, 2007, pp. 139–144.
- [14] J. Vaidya, V. Atluri, and J. Warner, "Roleminer: Mining roles using subset enumeration," in *Proceedings of the 13th ACM Conference on Computer and Communications Security*, 2006, pp. 144–153.
- [15] H. Lu, J. Vaidya, and V. Atluri, "Optimal boolean matrix decomposition: Application to role engineering," in *2008 IEEE 24th international conference on data engineering*, IEEE, 2008, pp. 297–306.
- [16] I. Molloy, N. Li, T. Li, Z. Mao, Q. Wang, and J. Lobo, "Evaluating role mining algorithms," in *Proceedings of the 14th ACM Symposium on Access Control Models and Technologies*, 2009, pp. 95–104.
- [17] A. Ene, W. Horne, N. Milosavljevic, P. Rao, R. Schreiber, and R. E. Tarjan, "Fast exact and heuristic methods for role minimization problems," in *Proceedings of the 13th ACM Symposium on Access Control Models and Technologies*, 2008, pp. 1–10.
- [18] F. Zhu, C. Yang, L. Zhu, and J. Gu, "Application of Matrix Factorization Role Mining Algorithm in Role-Based Access Control for Edge RMP," in *2024 9th International Conference on Electronic Technology and Information Science (ICETIS)*, 2024, pp. 761–767.
- [19] C. Blundo and S. Cimato, "Role mining under user-distribution cardinality constraint," *Journal of information security and applications*, vol. 78, p. 103 611, 2023.
- [20] Q. Guo and M. Tripunitara, "The Secrecy Resilience of Access Control Policies and Its Application to Role Mining," in *Proceedings of the 27th ACM Symposium on Access Control Models and Technologies*, Association for Computing Machinery, 2022, pp. 115–126.
- [21] L. Dong, T. Wu, W. Jia, B. Jiang, and X. Li, "Computable Access Control: Embedding Access Control Rules Into Euclidean Space," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 53, no. 10, pp. 6530–6541, 2023.